

Assignment Part A

[Total: 100 marks]

Due date: 20 April, 2025.

Due time: 23:59 (link will be disabled automatically at 00:10, 21 April)

Advices:

- Submit **at least 30 minutes before** the due time to avoid any technical issues.
- If the submission fails due to a last-minute attempt (e.g. 10 minutes before due time) and avoidable issues such as network latency, resulting in the submission link being disabled, please email the zip file to **sorkv@utar.edu.my**. This may be considered a late submission.

Submission:

There are **TWO (2)** files to be submitted:

1. One softcopy report (in **.docx** format, named as **leadername.docx**). Use the report cover page provided. Assignment free riders should be reported and excluded from the report submission.
2. The program source file for Task 4, **warehouse_solution.c**

Group leader is to:

- place the two files in a folder name **GroupNumber_LeaderName** (e.g., G1_SorKeanVee).
- zip the folder.
- submit the zip file via the submission link provided in WBLE.

This part of assignment is to be **evaluated as a group**. All the tasks are independent and the tasks should be distributed to group member.

Number of members: Maximum 4 students, not limited to the same programme nor tutorial group.

Objective:

To expose students to

- Logical Volume Management (LVM)
- Storage quota configuration
- RAID configuration
- Threads synchronisation and mutual exclusion

Tool and Environment:

Create a virtual machine (in VirtualBox or VMware Workstation) with the following specification:

Name of virtual machine: ***your_name_vm***

Processor: **2**

Memory: **2GB** (can be less or more based the available memory in the host machine)

HD storage: **20 GB**

Recommended OS: **Lubuntu 22.04**

User name: ***Your name***

Host (machine) name: **osvm**

NOTE: Using Kali Linux virtual machine image given in another course does not count as “installation”.

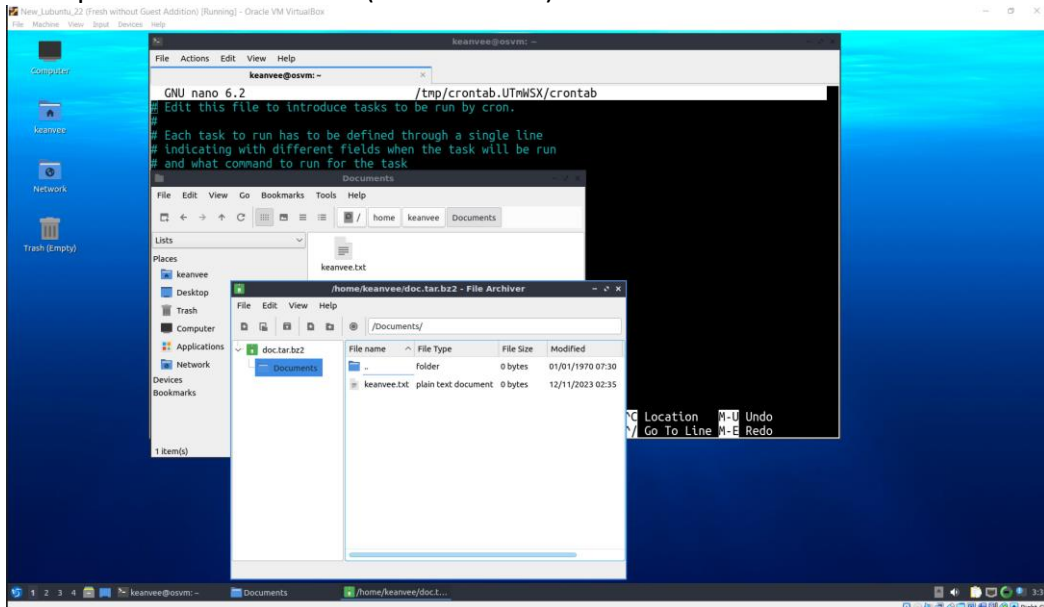
Reminder:

All the screenshots in the report must contain / able to show your user name to **proof the originality of your work**. Notice that the screenshots included in this document show my user name “keanvee”.

Any group reports that have the **same screenshots or usernames**, will be counted as **plagiarism** and be awarded 0 marks.

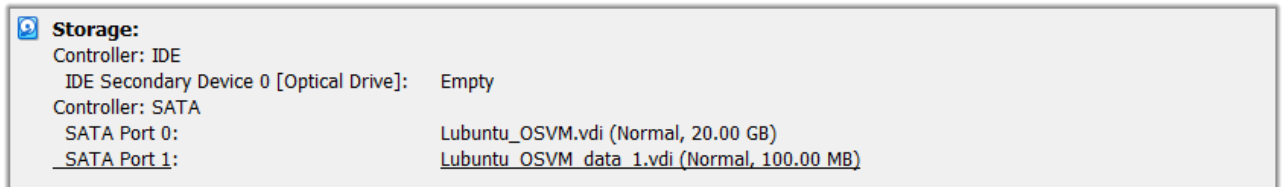
The content of all screenshots must be clear enough to be read. DO NOT take the screenshot of the entire screen.

Example of bad screenshot (entire screen):



Task 1: Configure storage quota**(20 marks)**

- Install the required packages:
 - quota
 - quotatool
- Make sure that the VM is shut down (not saved state).
- Add a 100 MB hard disks in the VM settings. The following screenshot shows an example of the VM's settings in VirtualBox with 2 additional hard disks added. VMware Workstation Player may look different.



- Make sure the newly added hard disk is detected in the VM. Screenshot in Figure 1 shows the newly added hard disk named as **sdb**. Your newly added hard disk might be named differently if there are more than 2 hard disks in the VM.

NAME	FSTYPE	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINT	SIZE
sda							20G
└sda1	ext4		87cd6fa8-a5d6-42c4-b490-803721a72ceb	12.3G	32%	/	20G
sdb							100M
sr0							1024M

keanvee@osvm: ~\$

Figure 1

NOTE:

You may want to take a snapshot of the VM before proceed to the next step.

- Create a primary partition using the entire disk space and **ext4** filesystem on the disk by using commands, with the following requirements.
 - Use GPT partition table
 - Use the entire disk space to create the partition
 - Create the ext4 filesystem on the partition

Figure 2 shows the example of partition with ext4 filesystem created on it.

NAME	FSTYPE	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINT	SIZE
sda							20G
└sda1	ext4		87cd6fa8-a5d6-42c4-b490-803721a72ceb	12.3G	32%	/	20G
sdb							100M
└sdb1	ext4		32ffcf01-7776-4ef5-8a7f-6d2500a097c5				99M
sr0							1024M

keanvee@osvm: ~\$

Figure 2

CAUTION:

It is recommended to create a snapshot of the VM or backup the `/etc/fstab` file before proceed to edit `/etc/fstab` because a small mistake such as additional character in the file may cause the OS fails to load after reboot.

- Configure `/etc/fstab` to mount the newly created partition to the mount point `/mnt/data` and enable the **user quota**.

- Mount the partition to /mnt/data to apply the quota configuration. Figure 3 shows the partition is mounted to /mnt/data

NAME	FSTYPE	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINT
sda						
└sda1	ext4		87cd6fa8-a5d6-42c4-b490-803721a72ceb	12.3G	32%	/
sdb						
└sdb1	ext4		32ffcf01-7776-4ef5-8a7f-6d2500a097c5	84.9M	0%	/mnt/data
sr0						

Figure 3

- Configure quota for studentXXX where XXX are the last 3 digits of your student ID:
 - Soft limit: 50 MB
 - Hard limit: 60 MB

Figure 4 shows an example of quota configured for a user named **tester001** on a logical volume.

keanvee@osvm: ~									
GNU nano 6.2									
Disk quotas for user tester001 (uid 1001):									
Filesystem	blocks	soft	hard	inodes	soft	hard	inodes	soft	hard
/dev/mapper/vg--data-lv--backup	61440	50M	60M	3	0	0	0	0	0

Figure 4

- Create a directory for user **studentXXX** in /mnt/data and change the ownership of the directory to studentXXX and its respective group. The default owner of the directory is root. Figure 5 shows an example of the ownership and group of the studentXXX directory have been changed.

```
keanvee@osvm:~$ ll /mnt/data/
total 36
drwxr-xr-x 4 root      root      4096 Mac   8 03:43 ./
drwxr-xr-x 3 root      root      4096 Mac   8 02:40 ../
-rw----- 1 root      root      7168 Mac   8 03:59 aquota.user
drwx----- 2 root      root     16384 Mac   7 18:50 lost+found/
drwxr-xr-x 2 student123 student123 4096 Mac   8 03:43 student123/
```

Figure 5. Ownership and group of the directory **student123** changed to *student123*

- Remove all the permissions (read, write, execute) granted to the group and others so that only the owner (studentXXX) is granted with all the permissions.
- Login as studentXXX and perform the following operations to **mnt/data/studentXXX**
 - Copy a file that is larger than 50 MB but less than 60 MB.
 - Remove the file copied in step (i).
 - Copy a file that is larger than 60 MB. An error message should be displayed, refer to Figure 7 for example.

UCCD2103 Operating Systems

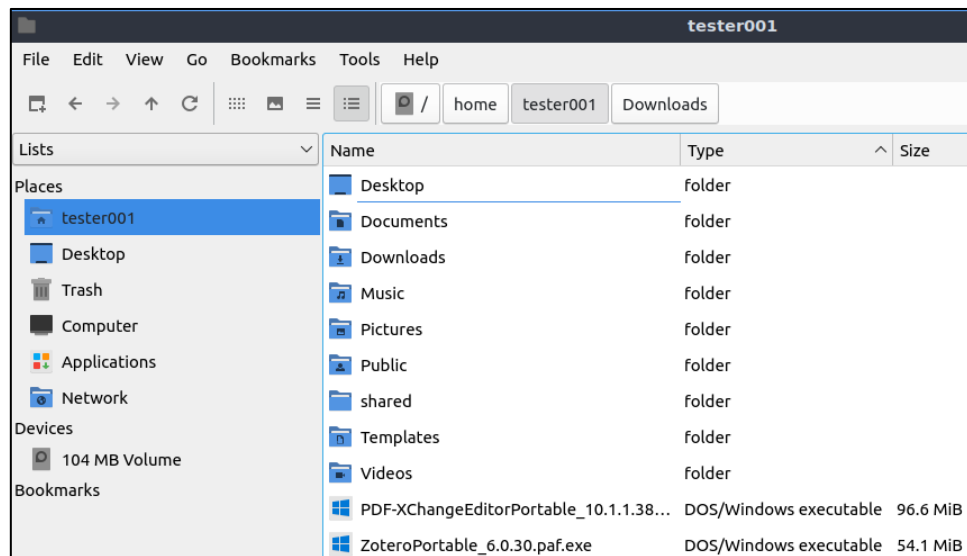


Figure 6

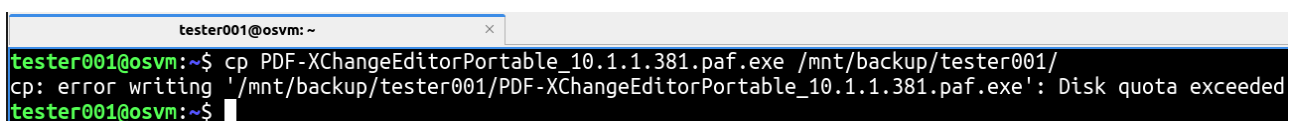


Figure 7

To be included in the report:

1. List the steps to create partition and file system in the additional disk. Include the relevant command(s) used in each step.
2. List the steps to configure and apply the quota for studentXXX. Include the relevant command(s) used in each step.
3. The following screenshots, should be included in the report.
 - The content of /etc/fstab with the entry for /mnt/data
 - Quota configuration for studentXXX. Refer to Figure 4.
 - A screenshot that shows a file to be copied to /mnt/data/studentXXX is larger than 60 MB. Refer to Figure 6.
 - A screenshot that shows the error message in attempt to copy a file that exceeds the quota to /mnt/data/studentXXX. Refer to Figure 7.

Task 2: Logical Volume Management (LVM) - Configure Logical Volume**(20 marks)**

- Make sure that the VM is shut down (not saved state).
- Add 2 hard disks (size of 60 MB and 100 MB each) in the VM settings.
- Start the VM and make sure the 2 hard disks are detected in the VM. You may use a GUI partition manager in this task as in Figure 8.

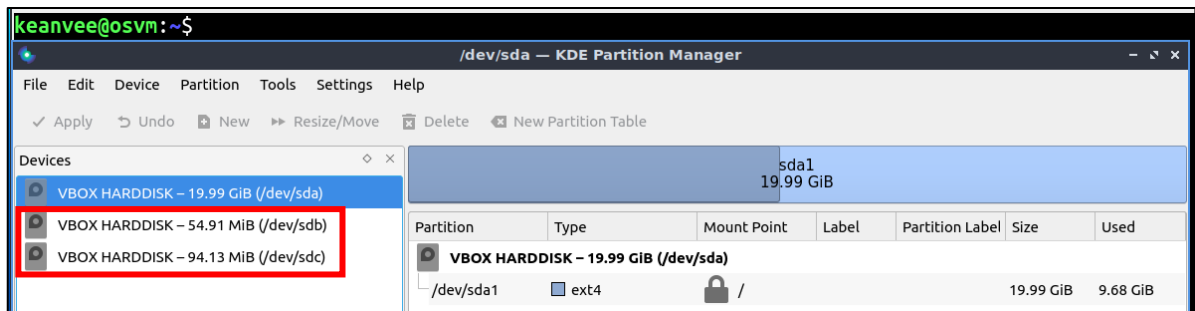


Figure 8

- Create a volume group named **vg-recovery** using the 60 MB hard disk.
- Create a logical volume named **lv-user** in the volume group using the entire space available in the volume group. E.g., if the size of volume group is 50 MB, the logical volume size will be 50 MB too. Figure 9 shows an example of logical volume created. Notice that the size of logical volume is equal to the size of volume group.

```

VG          #PV #LV #SN Attr   VSize VFree
vg-recovery 1   1   0 wz--n- 56.00m 0
keanvee@osvm:~$

LV          VG          Attr      LSize  Pool Origin Data%
lv-user     vg-recovery  -wi-a----- 56.00m
keanvee@osvm:~$

```

Figure 9

- Before a logical volume can be used, a filesystem must be created on that logical volume. Create the **ext4** file system on the logical volume **lv-user**.
- Mount the logical volume to the mount point **/mnt/lvm**
Figure 10 shows the filesystem **ext4** has been created and the logical volume is mounted to the mount point **/mnt/lvm**

```

keanvee@osvm: ~
sdb LVM2 m LVM2 I1c47W-n0ZV-hvMw-TYU5-n0x5-aC97-wxYPHW
└vg--recovery-lv--user
  ext4 1.0 835b4eaa-f7b4-4976-839b-907da8b7653f 44.5M 0% /mnt/lvm
sdc
sr0
keanvee@osvm:~$

```

Figure 10

- Try to copy a file that exceeds the logical volume size. Error message will be displayed.

```

keanvee@osvm:~$ sudo cp Downloads/ZoteroPortable_6.0.30.paf.exe /mnt/lvm/
[sudo] password for keanvee:
cp: error writing '/mnt/lvm/ZoteroPortable_6.0.30.paf.exe': No space left on device
keanvee@osvm:~$ df -h /mnt/lvm/
Filesystem                                Size  Used Avail Use% Mounted on
/dev/mapper/vg--recovery-lv--user         49M   48M    0 100% /mnt/lvm

```

Figure 11

- Remove the file from /mnt/lvm/
- Perform the following operations to the 100 MB hard disk
 - Create a GPT partition table
 - Create a primary partition of 50 MB with ext4 filesystem
 - Create a primary partition using the remaining space with ext4 filesystem

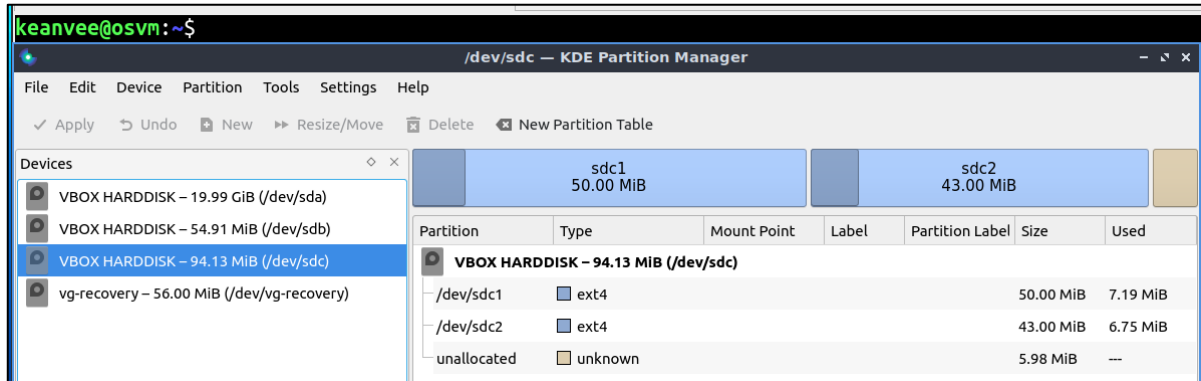


Figure 12. Summary of the partitions created

- Extend the logical volume **lv-user** to the 50 MB partition (/dev/sdc1 in the example in Figure 12)
- Try to copy the same file again and it should succeed if the logical volume is extended correctly.

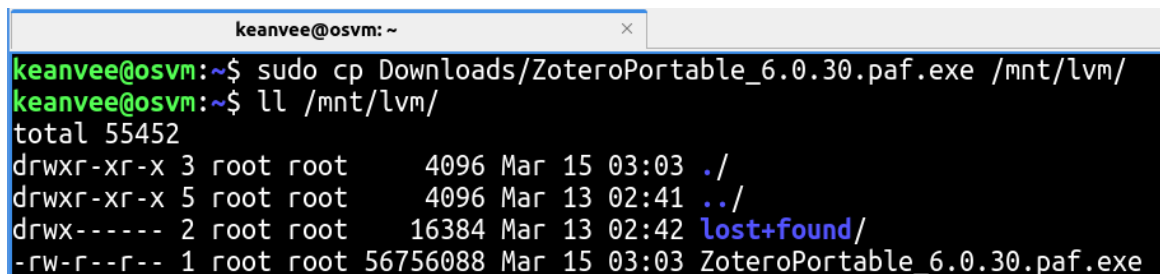


Figure 13. File copied successfully after the logical volume is extended

To be included in the report:

1. List the steps to create and mount the logical volume to a mount point. Include the relevant command(s) used in each step. You may screenshot the command used.
2. The following screenshots, should be included in the report.
 - A screenshot that shows two additional hard disks have been added. Refer to Figure 8 as an example.
 - The information of the logical volume created, refer to Figure 9 as an example.
 - A screenshot to show the logical volume is mounted to the mount point **/mnt/lvm**. Refer to Figure 10 as an example.
 - A screenshot that shows error writing files to the logical volume. Refer to Figure 11 as an example.
 - A screenshot that shows additional partition has been added to volume group. Example:

```
PV          VG          Fmt Attr PSize PFree
/dev/sdb    vg-recovery  lvm2 a--  56.00m  0
/dev/sdc1    vg-recovery  lvm2 a--  48.00m  0
keanvee@osvm:~$
```

UCCD2103 Operating Systems

- Screenshots that show the logical volume has been expanded. Example:

Before:

```
LV      VG      Attr      LSize  Pool Origin Data%  
lv-user vg-recovery -wi-a----- 56.00m  
keanvee@osvm:~$
```

After:

```
LV      VG      Attr      LSize  Pool Origin Data%  
lv-user vg-recovery -wi-a----- 104.00m  
keanvee@osvm:~$
```

- A screenshot that shows the file can be copied successfully. Refer to Figure 13 as an example.

Task 3: RAID Configuration**(15 marks)**

To accomplish this task, add 4 additional 50 MB hard disks to VM and install the **mdadm** package to manage and configure RAID.

Configure a disk array based on the following guidelines.

- Create a RAID level 5 named **/dev/md0**, with the first 3 additional disks. The following screenshot shows the RAID is created and started successfully.

```
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
keanvee@osvm:~$
```

Figure 14. Disk array is created and started

- Display the information of **/dev/md0** as in the following screenshot.

```
/dev/md0:
  Version : 1.2
  Creation Time : Wed Mar 19 00:27:42 2025
  Raid Level : raid5
  Array Size : 98304 (96.00 MiB 100.66 MB)
  Used Dev Size : 49152 (48.00 MiB 50.33 MB)
  Raid Devices : 3
  Total Devices : 3
  Persistence : Superblock is persistent

  Update Time : Wed Mar 19 00:27:43 2025
  State : clean
  Active Devices : 3
  Working Devices : 3
  Failed Devices : 0
  Spare Devices : 0

  Layout : left-symmetric
  Chunk Size : 512K
  Consistency Policy : resync

  Name : osvm:0 (local to host osvm)
  UUID : 93255465:8abefa95:3446d52e:fac699e9
  Events : 18

   Number   Major   Minor   RaidDevice State
    0         8       32         0   active sync  /dev/sdc
    1         8       48         1   active sync  /dev/sdd
    3         8       64         2   active sync  /dev/sde

keanvee@osvm:~$
```

Figure 15. Information of the disk array

- Create ext4 filesystem on **/dev/md0**

```
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 24576 4k blocks and 24576 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done

keanvee@osvm:~$
```

- Mount **/dev/md0** to **/mnt/rd5** and display the space usage. You may copy some files to **/mnt/rd5** for testing.

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/md0        89M   24K   83M   1% /mnt/rd5
keanvee@osvm:~$
```

Figure 16. Mount point and space usage

- Add the fourth additional hard disk to **/dev/md0**. This hard disk will be added as a spare disk.

```
/dev/md0:
Version : 1.2
Creation Time : Wed Mar 19 00:27:42 2025
Raid Level : raid5
Array Size : 98304 (96.00 MiB 100.66 MB)
Used Dev Size : 49152 (48.00 MiB 50.33 MB)
Raid Devices : 3
Total Devices : 4
Persistence : Superblock is persistent

Update Time : Wed Mar 19 00:59:03 2025
State : clean
Active Devices : 3
Working Devices : 4
Failed Devices : 0
Spare Devices : 1

Layout : left-symmetric
Chunk Size : 512K
Consistency Policy : resync

Name : osvm:0 (local to host osvm)
UUID : 93255465:8abefa95:3446d52e:fac699e9
Events : 19

Number Major Minor RaidDevice State
0        8       32        0 active sync /dev/sdc
1        8       48        1 active sync /dev/sdd
3        8       64        2 active sync /dev/sde
4        8       80        - spare   /dev/sdf
keanvee@osvm:~$
```

Figure 17. Additional disk is added as a spare disk

NOTE:

If one of the three active disks fails, the spare disk will be used to replace the failed disk and the data recovery process will be started immediately.

- Given that the second active disk (sdd in Figure 17, your second active disk might be different) fails, mark it as failed and display the information of **/dev/md0**

```
mdadm: set /dev/sdd faulty in /dev/md0
keanvee@osvm:~$
```

The second disk is marked as failed.

UCCD2103 Operating Systems

```
/dev/md0:
  Version : 1.2
  Creation Time : Wed Mar 19 00:27:42 2025
  Raid Level : raid5
  Array Size : 98304 (96.00 MiB 100.66 MB)
  Used Dev Size : 49152 (48.00 MiB 50.33 MB)
  Raid Devices : 3
  Total Devices : 4
  Persistence : Superblock is persistent

  Update Time : Wed Mar 19 01:35:35 2025
  State : clean
  Active Devices : 3
  Working Devices : 3
  Failed Devices : 1
  Spare Devices : 0

  Layout : left-symmetric
  Chunk Size : 512K

Consistency Policy : resync

        Name : osvm:0 (local to host osvm)
        UUID : 93255465:8abefa95:3446d52e:fac699e9
        Events : 38

   Number   Major   Minor   RaidDevice State
    0         8       32         0      active sync  /dev/sdc
    4         8       80         1      active sync  /dev/sdf
    3         8       64         2      active sync  /dev/sde

    1         8       48         -      faulty   /dev/sdd

keanvee@osvm:~$
```

Figure 18. The spare disk has replaced the failed disk

- Remove the failed disk from the disk array.

```
mdadm: hot removed /dev/sdd from /dev/md0
keanvee@osvm:~$
```

```
/dev/md0:
  Version : 1.2
  Creation Time : Wed Mar 19 00:27:42 2025
  Raid Level : raid5
  Array Size : 98304 (96.00 MiB 100.66 MB)
  Used Dev Size : 49152 (48.00 MiB 50.33 MB)
  Raid Devices : 3
  Total Devices : 3
  Persistence : Superblock is persistent

  Update Time : Wed Mar 19 01:43:03 2025
  State : clean
  Active Devices : 3
  Working Devices : 3
  Failed Devices : 0
  Spare Devices : 0

  Layout : left-symmetric
  Chunk Size : 512K

Consistency Policy : resync

        Name : osvm:0 (local to host osvm)
        UUID : 93255465:8abefa95:3446d52e:fac699e9
        Events : 39

   Number   Major   Minor   RaidDevice State
    0         8       32         0      active sync  /dev/sdc
    4         8       80         1      active sync  /dev/sdf
    3         8       64         2      active sync  /dev/sde

keanvee@osvm:~$
```

Figure 19. The failed disk has been removed

- Unmount the disk array and stop it.



```
mdadm: stopped /dev/md0
keanvee@osvm:~$
```

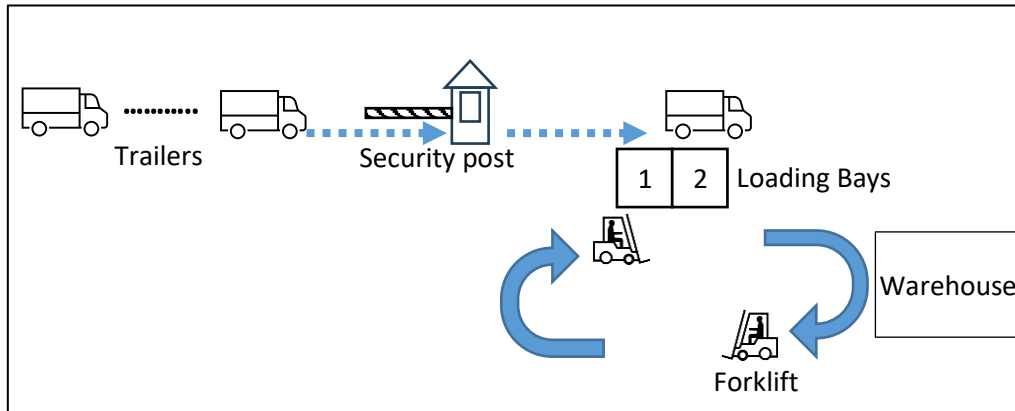
Figure 20. Disk array has been stopped

To be included in the report:

1. Specify the relevant command(s) used in each configuration step in the guidelines. You may screenshot the command(s) used.
2. The following screenshots, should be included in the report. The screenshot must show the **command used and its output**. Relevant screenshot can be placed in each configuration step.
 - A screenshot that shows the disk array is started. Refer to Figure 14.
 - A screenshot that shows information as in Figure 15.
 - A screenshot that shows disk usage as in Figure 16.
 - A screenshot that shows spare disk is added as in Figure 17.
 - A screenshot that shows the second active disk is marked as failed as in Figure 18.
 - A screenshot that shows the failed disk has been removed as in Figure 19.
 - A screenshot that shows the disk array has stopped as in Figure 20.

Task 4: Synchronisation with semaphores**(40 marks)**

The figure below shows the operation in a small warehouse with 2 loading bays. Trailers will arrive from time to time and each trailer must be checked by the security officer upon arrival. Once the trailer has been checked, it may proceed to the loading area and waits for available loading bay. If a loading bay is available, the trailer can park at that bay and unload the containers. There are 2 forklifts to move the containers to the warehouse.



The operations of trailers, security and forklifts can be simulated by using threads based on the following guidelines.

- Each trailer, security and forklift will be simulated by a separate thread.
- Assume that
 - a trailer will arrive every 3-4 seconds.
 - total number of trailers will arrive is within 3 to 10, inclusive.
 - each trailer carries 2 containers and requires 2-4 seconds to unload the containers.
 - the security officer takes 2-3 seconds to check a trailer.
 - a forklift takes 3 seconds to move a container to warehouse.
 - if a forklift has idled for 10 seconds, it is safe to stop and quit.
- Each thread should print a message to indicate its current operation. Refer to Figure 21 as an example.
- The program should be able to accept an argument that indicates the number of trailers arriving. Figure 21 shows the value **3** is passed to a program named **f3** as an argument in the command line.

Implement the operations described above using POSIX C programming language with pthread for multithreading and semaphores for threads synchronisation. The source codes should be properly commented, and meaningful name should be used for any variable or function. Please implement error handlings in thread creation and semaphore initialisation.

Name the source file as **warehouse_solution.c** (This file is to be submitted)

The following screenshot show the example of output when the program is executed. Note that the output might be different in every execution.

```

keanvee@osvm:~$ ./f3 3
Total number of trailers: 3
Trailer-1: Arrived.
Trailer-1: Under checking...
Security: Standby
Security: Checking trailer-1
Security: Checked & Released
Security: Standby
Trailer-1: Waiting for loading bay...
Trailer-1: Total containers = 2
Trailer-2: Arrived.
Trailer-2: Under checking...
Security: Checking trailer-2
Security: Checked & Released
Security: Standby
Trailer-2: Waiting for loading bay...
Trailer-2: Total containers = 4
Trailer-3: Arrived.
Trailer-3: Under checking...
Security: Checking trailer-3
Trailer-1: Unloaded. Leaving...
Trailer-2: Unloaded. Leaving...
Forklift-1: Waiting for containers
Forklift-1: Moving a container
Forklift-1: Remaining = 3
Forklift-2: Waiting for containers
Forklift-2: Moving a container
Forklift-2: Remaining = 2
Trailer-3: Waiting for loading bay...
Trailer-3: Total containers = 4
Trailer-3: Total containers = 4
Security: Checked & Released
Security: Exit
Security joined
Forklift-1: Moving a container
Forklift-1: Remaining = 3
Forklift-2: Moving a container
Forklift-2: Remaining = 2
Trailer-3: Unloaded. Leaving...
All trailers joined
Forklift-1: Moving a container
Forklift-1: Remaining = 1
Forklift-2: Moving a container
Forklift-2: Remaining = 0
Forklift-1: Time out. Exit
Forklift-2: Time out. Exit
All forklifts joined
Program terminates.
keanvee@osvm:~$

```

Figure 21. Example of status messages from security, trailers and forklifts

To be included in the report:

- Content of warehouse_solution.c
- A screenshot that shows the output of the execution of the program (refer to Figure 21)

Report writing, formatting and tidiness.

(5 marks)

Appendix 1: Sampil pthread program

The **count_char.c** is a program that will create a thread to count the number of characters entered by user.

- The main thread will create a thread (referred as worker thread) to execute the `thread_function`.
- The main thread will read user input until "end" is entered.
- When "end" is entered, the main thread will wait for the worker to exit before it exits.
- Once user has entered text (ENTER is pressed), the main thread performs signal operation on semaphore.
- When the worker thread starts, it will be blocked by the wait operation on semaphore (wait for main thread's signal).
- When the worker thread is unblocked, it will count the number of characters entered by user (stored in `work_area[ ]`).

To GNU C compiler (gcc) will be used to compile the program, install it if it is not installed. The pthread library / package is installed by default in Ubuntu/Ubuntu.

Command to compile the program:

```
gcc -o count_ch count_char.c -lpthread
```

To execute the program:

```
./count_ch
```

Sample output:

```
keanvee@osvm:~$ ./count_ch
Input some text. Enter 'end' to finish
line 1.
You input 7 characters
Another line.
You input 13 characters
end

Waiting for thread to finish...
Thread joined
keanvee@osvm:~$
```

count_char.c

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <semaphore.h>

void *thread_function(void *arg);

sem_t bin_sem;

#define WORK_SIZE 1024

char work_area[WORK_SIZE];

int main() {
    int res;
    pthread_t a_thread;
    void *thread_result;

    res = sem_init(&bin_sem, 0, 0);
    if (res != 0) {
        perror("Semaphore initialization failed");
        exit(EXIT_FAILURE);
    }

    res = pthread_create(&a_thread, NULL, thread_function, NULL);
    if (res != 0) {
        perror("Thread creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Input some text. Enter 'end' to finish\n");
    while(strncmp("end", work_area, 3) != 0) {
        fgets(work_area, WORK_SIZE, stdin);
        sem_post(&bin_sem);
    }
    printf("\nWaiting for thread to finish...\n");
    res = pthread_join(a_thread, &thread_result);
    if (res != 0) {
        perror("Thread join failed");
        exit(EXIT_FAILURE);
    }
    printf("Thread joined\n");
    sem_destroy(&bin_sem);
    exit(EXIT_SUCCESS);
}

void *thread_function(void *arg) {
    sem_wait(&bin_sem);
    while(strncmp("end", work_area, 3) != 0) {
        printf("You input %d characters\n", strlen(work_area) - 1);
        sem_wait(&bin_sem);
    }
    pthread_exit(NULL);
}

```

----- End of document -----